

Abhigyan Dey

Toronto, ON | ed.dey@mail.utoronto.ca | [GitHub](#) | [Portfolio](#)

EXPERIENCE

AI Engineer

Apr 2026 – Aug 2026

Government Technology Project, Remote

- Shipped AI-enabled applications and automation workflows in Python (REST APIs, LLMs, retrieval, structured data), turning ambiguous requirements into working software through rapid iteration on user feedback.
- Built AI orchestration infrastructure and agentic workflows combining model reasoning with tools and knowledge sources.
- Delivered end-to-end RAG pipelines spanning ingestion, embeddings, retrieval and generation.
- Established evaluation, validation and deployment pipelines for agents to surface failures and edge cases early.

AI/ML Model Evaluator & Code Reviewer

Nov 2024 – Present

Outlier AI, Mercor & Feather, Remote

- Reviewed AI-generated code at high volume for correctness, edge cases and reasoning, giving structured written feedback.
- Reproduced bugs and isolated root causes in Python and SQL; evaluated LLM outputs for a third party's retrieval layer.

TECHNICAL SKILLS

AI & Agents: LLM APIs (Claude, OpenAI), agent orchestration (LangGraph), tool calling, structured outputs (Pydantic), RAG, embeddings, prompt and context engineering, evaluation harnesses, human-in-the-loop design

Languages: Python, TypeScript/JavaScript (Node.js), SQL, C++, Java

Backend & Data: FastAPI, REST APIs, PostgreSQL, Redis, ChromaDB, pandas, NumPy, ETL and data validation, idempotent and fault-tolerant service design

Tooling & Cloud: Docker, Git, CI/CD, pytest, AWS, Azure, Claude Code, Cursor, GitHub Copilot

EDUCATION

University of Toronto, St. George

Honours Bachelor of Science, Computer Science, Mathematics & Statistics

Activities: CS Student Union (member); RSG Leader

Toronto, ON

Sep 2022 – Aug 2026

PROJECTS

Agentic Negotiation Platform (Agentic Dating MVP)

Node.js, Claude API, Apify | [GitHub](#)

- Designed, built and launched a multi-agent platform where each user's autonomous agent reads their public profile, negotiates compatibility with every other agent and returns a ranking; live with 20 real users; B2B MVP.
- Built a directional compatibility model (needs, interests, traits, location, intent) driving a three-stage agent dialogue; each agent files a verdict for its own human, and rankings blend both sides' verdicts.
- Engineered for traceability: every extracted claim keeps its source quote and agent turns log reasoning.
- Shipped a zero-dependency Node.js service with tiered ingestion fallbacks and a no-LLM-key path.

Financial RAG

Python, FastAPI, ChromaDB, OpenAI Embeddings, Claude, Streamlit | [GitHub](#)

- Built a seven-stage RAG pipeline for financial documents: PDF loading with page tracking, paragraph-aware chunking, batched embeddings, Chroma storage and cosine retrieval with normalized scores.
- Grounded Claude to answer only from retrieved context, cite source and page inline, and say when context is insufficient.
- Made ingestion idempotent (re-ingesting overwrites, never duplicates), with custom error types and smoke tests per stage.
- Exposed it via FastAPI (upload, query, streaming, Swagger docs) with a Streamlit chat UI as a separate HTTP client.

Invoice Exception Agent (in progress)

Python, LangGraph, FastAPI, PostgreSQL, Docker | [GitHub](#)

- Architecting a LangGraph pipeline (extract, validate, repair, three-way match, policy check, route) that auto-posts clean invoices and sends the rest to a human with the evidence attached.
- Design decisions: code validates before any LLM judges, repair loop capped at two, default route is review not approve, and Postgres checkpoints let an approval wait days and survive a restart.
- Idempotent ERP posting via a unique key, supervisor agents confined to exceptions, and an audit log on every exit path.
- Specified an eval plan on seeded faults (wrong total, duplicate, price mismatch) scored by code against ground truth.

Reflexion Coding Agent

Python, LLM APIs | [GitHub](#)

- Implemented the NeurIPS 2023 Reflexion paper from scratch: actor, sandboxed evaluator, reflector, memory.
- Built a 20-problem benchmark (single-shot, blind retry, reflection) at an identical trial budget to isolate reflection.